

Lunar Lander

احمد رضا توسلی بهابادی

مقدمه و هدف پروژه

در این پروژه، هدف پیاده‌سازی و آموزش یک عامل هوشمند برای کنترل و فرود موفقیت‌آمیز فضاپیما در محیط LunarLander است. این مسأله یکی از مسائل کلاسیک یادگیری تقویتی (Reinforcement Learning) محسوب می‌شود که در آن عامل باید با دریافت بازخورد از محیط، یاد بگیرد تا پاداش دریافتی را حداکثر کرده و با حفظ تعادل و مصرف بهینه سوخت، دقیقاً بین دو پرچم مشخص شده فرود آید. برای پیاده‌سازی این سیستم، از الگوریتم ارتقایافته‌ی Deep Q-Network (DQN) همراه با مکانیزم‌های تثبیت پیشرفته استفاده شده است.

۱ ساختار پروژه

پروژه پیاده‌سازی شده بر پایه‌ی ساختاری ماژولار و تفکیک شده طراحی شده است تا خوانایی، نگهداری و توسعه‌ی کد تسهیل گردد:

- **game.py**: ماژول رابط محیط بازی و مدیریت ارتباط با API محیط Gymnasium.
- **model.py**: تعریف ساختار معماری شبکه عصبی عمیق (PyTorch).
- **agent.py**: پیاده‌سازی منطق عامل هوشمند شامل شبکه سیاست، شبکه هدف، حافظه بازپخش و الگوریتم به‌روزرسانی نرم.
- **train.py**: مدیریت متمرکز هاپرپارامترها، حلقه اصلی آموزش، به‌روزرسانی وزن‌ها و ارزیابی توقف زودهنگام.
- **test.py**: اجرای مدل آموزش دیده جهت ارزیابی نهایی در شرایط تصادفی.

۲ مقایسه ورودی با مدل‌های DQN کلاسیک

در پیاده‌سازی‌های کلاسیک DQN (مانند مقاله سال ۲۰۱۳ و مقاله Nature 2015 در بازی‌های آتاری)، ورودی مدل به صورت فریم‌های تصویری با ابعاد $N \times M$ پیکسل بود. در آن ساختارها، جهت استخراج حالت و حذف ابعاد اضافه، تصویر ابتدا وارد یک تابع تبدیل ویژگی (مانند ϕ) می‌شد و سپس برای درک مفاهیمی مانند سرعت و جهت حرکت، ۴ فریم متوالی پشت سر هم قرار می‌گرفتند. در محیط LunarLander، نیازی به پردازش تصویر نیست؛ چرا که در هر گام زمانی، محیط یک بردار حالت ۸-مؤلفه‌ای به صورت مستقیم ارائه می‌دهد:

$$s = [x, y, v_x, v_y, \theta, \omega, c_1, c_2]^T \quad (1)$$

این بردار به صورت کامل شامل مختصات مکانی (x, y) ، مؤلفه‌های سرعت خطی (v_x, v_y) ، زاویه و سرعت زاویه‌ای سفینه (θ, ω) و دو متغیر باینری برای اعلام اتصال پایه‌های چپ و راست سفینه به سطح ماه (c_1, c_2) است.

۳ معماری شبکه عصبی Deep Q-Network

شبکه عصبی نگاشت‌دهنده حالت به مقادیر $Q(s, a)$ ، یک شبکه عصبی عمیق (Neural Network) با مشخصات زیر است:

- لایه ورودی: شامل ۸ نورون مطابق با ابعاد بردار وضعیت محیط.
- لایه‌های پنهان: دو لایه پنهان متوالی کاملاً متصل (Fully Connected)، هر یک شامل ۱۲۸ نورون همراه با توابع فعال‌ساز غیرخطی ReLU و مقداردهی اولیه وزن‌ها با روش Kaiming Normal.
- لایه خروجی: شامل ۴ نورون متناظر با مقادیر Q (Q -values) برای هر یک از اکشن‌های مجاز با مقداردهی اولیه Xavier Uniform.

در انتخاب این معماری، ابعاد ۱۲۸ نورون برای لایه‌های پنهان تعادلی بهینه ایجاد می‌کند؛ زیرا تعداد بیش از حد نورون‌ها موجب بیش‌برازش (Overfitting) شده و تعداد کم آن‌ها نیز باعث سادگی بیش از حد مدل و عدم یادگیری کامل پیچیدگی‌های محیط می‌گردد. همچنین استفاده از تابع فعال‌ساز ReLU به دلیل محاسبات سریع و جلوگیری از مسأله محو شدگی گرادیان (Vanishing Gradient)، پایداری و سرعت همگرایی شبکه را افزایش می‌دهد.

۴ مکانیزم‌های تثبیت آموزش در الگوریتم DQN

۱.۴ حافظه بازپخش تجربه (Experience Replay Buffer)

در الگوریتم‌های سنتی Q-Learning آنلاین، نمونه‌های دریافتی متوالی دارای همبستگی شدید زمانی (Temporal Correlation) هستند که منجر به واگرایی، انفجار گرادیان و گیر افتادن در حلقه‌های بازتاب مثبت می‌شود. جهت حل این چالش، یک حافظه بازپخش قرار داده شده است که ۵-تایی‌های تجربه را ذخیره می‌کند:

$$e_t = (s_t, a_t, r_t, s_{t+1}, \text{done}) \quad (۲)$$

در هر گام آموزش، نمونه‌برداری تصادفی از این حافظه انجام شده و سپس فرآیند پس‌انتشار گرادیان (Backpropagation) اجرا می‌شود تا همبستگی بین نمونه‌ها شکسته شود.

۲.۴ استراتژی اکتشاف و بهره‌برداری (ϵ -Greedy Policy)

برای ایجاد تعادل میان اکتشاف (Exploration) و بهره‌برداری (Exploitation)، از استراتژی ϵ -Greedy استفاده شده است. عامل با احتمال ϵ یک اکشن تصادفی و با احتمال $1 - \epsilon$ بهترین اکشن پیش‌بینی‌شده توسط شبکه را انتخاب می‌کند. مقدار ϵ در پایان هر اپیزود طبق فرمول نمایی زیر کاهش می‌یابد:

$$\epsilon_{t+1} = \max(\epsilon_{\min}, \epsilon_t \times \text{decay}) \quad (۳)$$

علت استفاده از تابع $\max$ و مقایسه آن با $\epsilon_{\min}$ این است که می‌خواهیم همواره یک حداقل درصد از اکتشاف ($\epsilon_{\min} = 0.01$) درون مدل باقی بماند تا عامل دانش خود را به کلی منجمد نکند.

۳.۴ حل مسأله هدف متحرک با بهروزرسانی نرم (Soft Target Update)

در پیاده‌سازی‌های کلاسیک DQN (مانند مقاله Nature 2015)، وزن‌های شبکه هدف (θ^-) ثابت نگه داشته شده و هر C گام یک‌بار به صورت سخت (Hard Update) با وزن‌های شبکه اصلی جایگزین می‌شدند ($\theta^- \leftarrow \theta$). این کپی‌برداری یک‌باره در بازه‌های زمانی مشخص، باعث تغییر ناگهانی مقادیر هدف بلن و در نتیجه ایجاد نوسانات و شوک‌های شدید در تابع زیان می‌گردید.

در این پروژه، از روش بهروزرسانی نرم (Soft Target Update) بر پایه میانگین‌گیری پولیاک (Polyak Averaging) استفاده شده است. در این مکانیزم، در هر گام زمانی، وزن‌های شبکه هدف به صورت پیوسته، هموار و تدریجی با ضریب مشخصی از وزن‌های شبکه اصلی ترکیب می‌شوند تا از شوک ناشی از تغییرات دوره‌ای جلوگیری شود:

$$\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^- \quad (۴)$$

که در آن $\tau \ll 1$ (مقدار $\tau = 0.001$) ضریب بهروزرسانی نرم است. این تغییر، تارگت بلن را به یک «هدف متحرک هموار» تبدیل کرده، شوک‌های ناشی از کپی دوره‌ای را حذف می‌کند و ضمن کاهش چشمگیر نوسانات تابع زیان، پایداری و سرعت همگرایی مدل را افزایش می‌دهد.

۵ روند کامل آموزش و ارزیابی الگوریتم

فرآیند آموزش عامل هوشمند در محیط LunarLander طی یک حلقه تکرار شونده و براساس گام‌های زیر انجام می‌شود:

۱.۵ ۱. مقداردهی اولیه (Initialization)

پیش از شروع حلقه آموزش، موارد زیر در حافظه برنامه بارگذاری می‌شوند:

- شبکه‌های عصبی: ایجاد شبکه سیاست (Policy Network) با وزن‌های اولیه θ و شبکه هدف (Target Network) با وزن‌های θ^- . در ابتدا وزن‌های شبکه هدف دقیقاً برابر با شبکه اصلی قرار داده می‌شوند ($\theta^- \leftarrow \theta$).

- حافظه بازپخش (Replay Buffer): ایجاد صف چرخشی با ظرفیت $N = 100,000$.

- هاپرپارامترها: تنظیم نرخ یادگیری $\alpha = 3 \times 10^{-4}$ ، ضریب تنزیل $\gamma = 0.99$ ، نرخ $\tau = 10^{-3}$ و حد نصاب توقف زودهنگام 240.0.

۲.۵ ۲. حلقه اصلی آموزش و برش زمانی گام‌ها (Step Truncation)

در هر اپیزود $e = 1, 2, \dots, E$ ، مراحل زیر طی می‌شوند:

۱. بازنشانی محیط: محیط راه‌اندازی شده و وضعیت اولیه s_0 دریافت می‌شود.

۲. حلقه گام‌های زمانی: در هر گام t ، اکشن a_t بر اساس سیاست Greedy- ϵ انتخاب شده، در محیط اجرا می‌شود و خروجی‌های $(s_{t+1}, r_t, done)$ دریافت می‌گردند.

۳. محدودیت زمان‌بندی (Step Truncation): جهت جلوگیری از رفتار معلق ماندن بی‌نهایت (Infinite Hovering) و اتلاف زمان آموزش، سقف گام زمانی هر اپیزود به 500 گام محدود شده است. اگر تعداد گام‌ها از این حد بگذرد، اپیزود خاتمه یافته و سیگنال done فعال می‌گردد.

۳.۵ ۳. استخراج دسته داده و بهروزرسانی پیوسته وزن‌ها

چنانچه داده‌های حافظه به اندازه اندازه دسته (Batch Size = 128) رسیده باشد، اقدامات زیر در هر گام زمانی اجرا می‌شوند:

۱. نمونه‌برداری تصادفی: استخراج یک دسته B -تایی از تجربه‌ها.

۲. محاسبه تارگت بلن (y_i) :

$$y_i = r_i + \gamma \cdot (1 - d_i) \cdot \max_{a'} Q_{\theta^-}(s'_i, a') \quad (۵)$$

۳. محاسبه تابع زیان و پس‌انتشار گرادیان: محاسبه خطای میانگین مربعات (MSE) و بهروزرسانی وزن‌های θ همراه با برش گرادیان (Gradient Clipping) با حد آستانه 10.0 برای جلوگیری از انفجار گرادیان:

$$L(\theta) = \frac{1}{B} \sum_{i=1}^B (y_i - Q_{\theta}(s_i, a_i))^2 \quad (۶)$$

۴. بهروزرسانی نرم شبکه هدف: اجرای بلافاصله‌ی معادله $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ در پایان هر گام آموزشی.

۴.۵ ۴. حد نصاب توقف زودهنگام بر اساس میانگین متحرک (Moving Average Early Stopping)

در انتهای هر اپیزود، میزان ϵ کاهش می‌یابد و میانگین متحرک پاداش ۱۰۰ اپیزود اخیر محاسبه می‌شود:

$$\bar{R}_{100} = \frac{1}{100} \sum_{k=e-99}^e R_k \quad (۷)$$

اگر $\bar{R}_{100} \geq 240.0$ باشد، فرآیند آموزش متوقف می‌گردد. انتخاب حد نصاب 240.0 (به جای حد نصاب استاندارد 200.0) تضمین می‌کند که عامل علاوه بر فرود بدون برخورد، مصرف سوخت و سرعت نشستن را به حالت بهینه رسانده است. همچنین تعیین این حد نصاب دقیق از بروز پدیده بیش‌برازش (Overfitting) جلوگیری می‌کند.

۵.۵ ۵. ارزیابی نهایی در شرایط تصادفی (Randomized Evaluation)

در مرحله ارزیابی (test.py)، وزن‌های ذخیره‌شده بارگذاری شده و مقدار $\epsilon = 0$ تنظیم می‌گردد. برای ارزیابی واقعی توانایی تعمیم‌پذیری عامل، تابع reset() محیط بدون مقداردهی اولیه ثابت (Fixed Seed) اجرا می‌شود تا عامل در شرایط مکانی و سرعتی کاملاً تصادفی و جدید مورد سنجش قرار گیرد.

۶ جدول هایپارامترهای بهینه‌شده پروژه

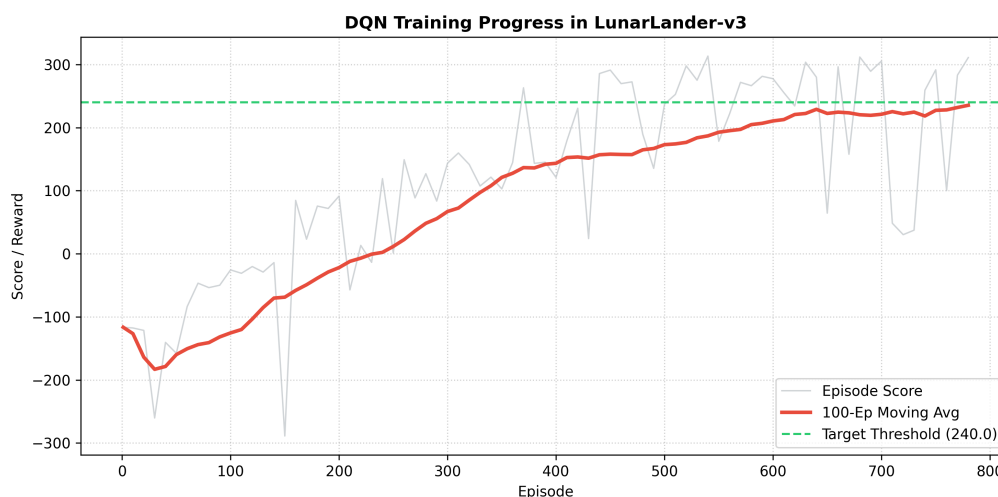
تمام هایپارامترهای کلیدی الگوریتم در یک دیکشنری متمرکز در فایل train.py پیکربندی شده‌اند که مقادیر بهینه آن‌ها در جدول ۱ خلاصه شده است.

جدول ۱: هایپرپارامترهای تنظیم شده برای الگوریتم DQN

نام هایپرپارامتر	توضیح عملکرد	مقدار بهینه
Total Episodes	حداکثر تعداد اپیزودهای آموزش	2500
Target Moving Avg	حد نصاب میانگین متحرک برای توقف زودهنگام	240.0
Max Steps per Episode	حداکثر گام مجاز در هر اپیزود (Truncation)	500
Hidden Dimension	تعداد نوروهای هر لایه پنهان شبکه	128
Learning Rate (α)	نرخ یادگیری بهینه‌ساز Adam	3×10^{-4}
Discount Factor (γ)	ضریب تنزیل پاداش‌های آینده	0.99
Soft Update Rate (τ)	ضریب به‌روزرسانی نرم شبکه هدف	0.001
Batch Size	اندازه دسته‌های نمونه‌برداری از حافظه	128
Buffer Capacity	حداکثر ظرفیت حافظه بازپخش تجربه	100,000
Epsilon Decay	ضریب کاهش نمایی نرخ اکتشاف در هر اپیزود	0.995
Epsilon Min	حد پایین نرخ اکتشاف ($\epsilon_{\min}$)	0.01

۷ نمودار روند آموزش الگوریتم

نمودار شکل ۱ روند تغییرات امتیازهای هر اپیزود و میانگین متحرک ۱۰۰ اپیزود اخیر را در طول فرآیند آموزش نشان می‌دهد. همان‌طور که در نمودار مشخص است، میانگین متحرک به صورت هموار و بدون نوسانات شدید به حد نصاب 240.0 رسیده است.



شکل ۱: منحنی یادگیری الگوریتم DQN و روند رشد میانگین متحرک پاداش‌ها

دلیل استفاده از حافظه تجربه

دلایل متفاوتی برای استفاده از حافظه تجربه وجود دارد. در الگوریتم DQN در هر مرحله آپدیت و استفاده از الگوریتم Backpropagation، از تعدادی از داده‌ها استفاده می‌کنیم. استفاده از حافظه تجربه باعث می‌شود بتوانیم از یک داده چند بار استفاده کنیم و حداکثر بهره را از اطلاعات به دست آمده ببریم.

دومین دلیل هم این است که در الگوریتم‌های آپدیت ضرایب شبکه عصبی، فرض می‌شود داده‌ها مستقل از یکدیگر هستند. اگر فقط از آخرین داده‌های به‌دست‌آمده استفاده کنیم و مثلاً همیشه ۱۰۰ داده‌ی آخر را برداریم، احتمال اینکه در حلقه بازتاب مثبت گیر کنیم زیاد است که در نهایت منجر به گیر کردن در بهینه‌های محلی می‌شود؛ چون مثلاً الگوریتم یک اکشن متوسط در یک مرحله پیدا می‌کند و این باعث می‌شود در Policy آن در مرحله‌ی بعد تأثیر بگذارد، چرا که از همان اکشن دقیقاً در آپدیت شبکه استفاده کرده‌ایم. در ادامه، این Policy تقویت شده و هر دفعه احتمال اینکه آن اکشن متوسط را انتخاب کنیم بیشتر می‌شود؛ در صورتی که شاید یک اکشن بسیار خوب دیگر در چند مرحله بعد و از یک راه دیگر قرار داشته باشد، اما این حلقه بازتاب مثبت یک جورهایی باعث کور شدن الگوریتم شده است.

دلیل استفاده از هدف ثابت و چالش هدف متحرک

در محاسبه‌ی مقدار هدف (y_i) از رابطه زیر استفاده می‌کنیم:

$$y_i = r_i + \gamma \max_{a'} Q(s'_i, a'; \theta) \quad (۸)$$

در فرآیند محاسبه‌ی گزاینان تابع زیان (Loss Function)، ما به دنبال به‌روزرسانی پارامترهایی هستیم که مقدار پیش‌بینی‌شده‌ی $Q(s_i, a_i)$ را به مقدار هدف (y_i) نزدیک‌تر کنند. اما چالش اصلی اینجاست که خود y_i نیز مقدار ثابتی نیست و درون فرمول خود، تابع Q را شامل می‌شود. بنابراین با هر بار تغییر و به‌روزرسانی مقدار Q ، مقدار هدف (y_i) نیز جابه‌جا می‌شود؛ دقیقاً مانند این است که به سمت هدفی حرکت کنیم که با هر قدم برداشتن ما به سمت آن، خود نیز جابه‌جا می‌شود.

البته عدم ثابت نگه داشتن y_i به این معنی نیست که مسئله مطلقاً غیرقابل حل باشد، اما احتمال مواجهه با واگرایی یا افزایش شدید تعداد گام‌های لازم برای همگرایی را به‌شدت افزایش می‌دهد. ثابت گرفتن تابع هدف (یا همان‌طور که در الگوریتم پیاده‌سازی شده یک گام فراتر رفتیم و تابع هدف را با سرعت کمتری به صورت نرم به‌روزرسانی کردیم) باعث می‌شود این تعقیب و گریز شکل نگرفته و فرآیند آموزش کاملاً پایدار گردد.

دلیل استفاده از اپسیلون و تفاوت مقادیر مختلف آن

متغیر اپسیلون به این دلیل وارد الگوریتم DQN می‌شود که ما بتوانیم همزمان با اینکه به سرچ در فضای حالت می‌پردازیم بتوانیم از تجربیات و سیاستی که تا الان یاد گرفتیم استفاده کنیم. اگر مقدار اپسیلون خیلی خیلی کم باشد یا با شیب زیادی کم بشود باعث میشود مدل میل زیادی به بهره‌برداری و استفاده از سیاستی که تا الان یاد گرفته داشته باشد. این احتمالاً مدل را دچار بایاس میکند چون به احتمال زیاد سیاست اولیه سیاست بهینه نیست و این مدل وقتی زیاد از سیاست یاد گرفته شده بهره‌برداری کند داده‌هایی هم که به حافظه تکرار می‌فرستد مربوط به همین سیاست هست و باعث میشود سیاست در گذر زمان احتمالاً تایید بشود، سیاستی که لزوماً بهینه نیست. از طرفی اگر اپسیلون با سرعت کمی کاهش یابد میل به تصادفی حرکت کردن مدل همچنان بالا خواهد بود و تایم ترین شدن مدل به طور نمایی بالا می‌رود. مدل از اشتباهاتش درس نمی‌گیرد و مثلاً حالت‌هایی که مرده هستند را چندین بار تکرار میکند و باعث سوخت شدن اپیزودها و دیتا می‌شود

تحلیل مقادیر ValueQ- و رفتار شبکه در شرایط سقوط

خروجی لایه آخر شبکه عصبی، یک بردار ۴-بعدی از مقادیر Q (Q-Values) برای حالت ورودی جاری s است. هر نورون خروجی، نشان‌دهنده امید ریاضی پاداش‌های تنزیل‌شده‌ی آتی به شرط انتخاب آن اکشن مشخص و پیروی از

سیاست عامل در گام‌های بعدی است:

$$Q(s, a) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right] \quad (9)$$

اگر سفینه در ارتفاع بالا با سرعت زیاد در حال سقوط آزاد باشد، منطقاً انتظار داریم مقدار Q -Value برای اکشن «روشن کردن موتور اصلی» (اکشن ۲) به مراتب بیشتر از سایر اکشن‌ها باشد ($Q(s, a_2) \gg Q(s, a_{\text{others}})$). در طول آموزش (Training)، مدل فهمیده که سقوط کردن امتیاز منفی (پنالتی) برای آن دارد. این موضوع باعث تغییر مقادیر Q در استیت‌های (States) نزدیک به زمین شده، وقتی که سفینه به سمت پایین می‌رود. در ادامه اپیزودهای آموزش، این پاداش منفی بین استیت‌های مجاور جابه‌جا شده و این‌طوری مدل به مرور زمان Q -Value استیت‌هایی را که در حال سقوط آزاد هستند، برای اکشن «روشن کردن موتور اصلی» خیلی بالا تخمین می‌زند.